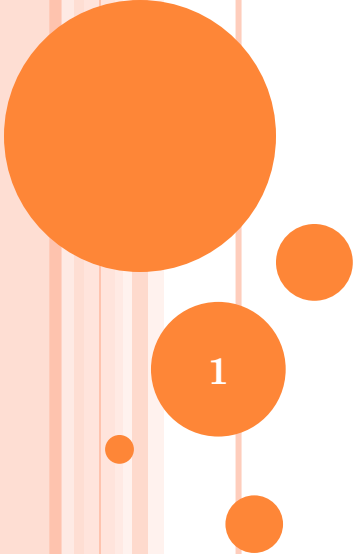




SISTEME DE OPERARE

Curs 4: Gestiunea memoriei externe

Gestiunea dispozitivelor de stocare (partea 2)

Kinga Márton

**Universitatea Tehnică din Cluj-Napoca
Departamentul de Calculatoare**

GESTIUNEA MEMORIEI EXTERNE

- În cuprins
 - Trecere în revistă a conceptelor din cursul precedent
 - Structura fizică a sistemelor de stocare secundare
 - Planificarea operațiilor de I/E
 - Gestiunea discului
 - RAID
 - SSD
 - Dispozitive de stocare terțiare

MEMORIA EXTERNĂ

- Utilizatorii au nevoie să stocheze date și aplicații
 - De **dimensiuni mari**
 - În mod **persistent** (durabil)
- Soluție
 - **Memoria externă** – sistemul de I/E
 - Discuri magnetice, discuri electronice, discuri optice, benzi magnetice, etc.
- Problemă
 - Timp de acces mare
 - Diferite tipuri de dispozitive de stocare
- Provocare
 - Organizarea eficientă a informației stocate persistent
 - Furnizarea unei **interfețe simple** și eficiente
 - Optimizarea operațiilor de I/E pentru concurență maximă
- Sisteme de fișiere
 - Mapare, acces, gestiune
 - **Colecție de fișiere** - Fiecare stochează date înrudite
 - **Structură de directoare** - Organizare și informații despre fișiere

FIȘIERUL

- Colecție de informații înrudite salvate pe memoria externă
- Unitatea logică de stocare
- Atribute (metadata)
 - Nume, Identificator, Tip, Locație, Dimensiune
 - Drepturi de acces (protecție), Timp, dată, identificare utilizator
- Informațiile despre fișiere sunt stocate în structura de directoare
 - **intrare în director** : Nume fișier + identificator
- Tipuri de fișiere
 - Fișiere obișnuite, Directoare, Fișiere legătură (Fizică, Simbolică), Fișiere speciale, Fișiere pipe
- Structura fișierelor
 - Fără structură
 - Structură specifică
- Operații asupra fișierelor
 - Creare, Scriere, Citire, Repoziționare în cadrul fișierului , Ștergere, Trunchiere + alte op.
- Metode de acces
 - Secvențial
 - Direct (aleator)

SISTEMUL DE FIȘIERE

- Directorul

- Fișier special gestionat de SO în mod transparent utilizatorului
- Mod de organizare a fișierelor
- Intrări în director – de regulă sunt asocieri nume – identificator

- Structura de directoare

- Structură pe un singur nivel
- Structură pe două niveluri
- Structură ierarhică

- Montarea SF

- Punct de montare

- Protecție

- Controlul accesului

- Decizii de proiectare

- interfața + implementare

- SF este organizat pe niveluri

- Fiecare nivel utilizează caracteristicile nivelului inferior pentru a crea funcționalități pentru nivelul superior

SISTEMUL DE FIȘIERE

○ Implementarea SF

- **Structurile de date** folosite pentru implementarea operațiilor sistemului de fișiere

• **Structuri stocate pe disc**

- Bloc de control pentru pornirea SO (per volum) - **boot block** sau **partition boot sector**
- Bloc de control al volumului (per volum) - **superblock** sau **master file table**
- **Structura de directoare** (per SF)
 - Pentru organizarea fișierelor
- **Bloc de control al fișierului** (FCB – File Control Block) (structura i-node)
 - Conține informații despre fișier – attribute, metadata

• **Structuri menținute în memorie**

- **Tabela de montare** - Informații despre fiecare volum montat
- **Structura de directoare – cache** - Info. despre directoare accesate recent
- **Tabela fișierelor deschise per sistem** - Include copii ale FCB pentru fișierele deschise
- **Tabela fișierelor deschise per proces** - Conține un pointer la intrarea corespunzătoare din tabela globală și alte info
- **Buffere** - Conțin date în transfer de la disc sau spre disc

IMPLEMENTAREA DIRECTOARELOR

○ Listă liniară

- Listă liniară de fișiere cu pointeri către blocurile de date

○ Tabele de dispersie

- Listă liniară + structură de dispersie
 - returnează un pointer către numele fișierului din lista liniară

○ Metode de alocare

- **Alocare contiguă**
- **Alocare prin înlănțuire**
- **Alocare prin indexare**

○ Gestiunea spațiului liber

- Hartă de biți
- Listă înlănțuită
- Listă cu grupare
- Contorizare

SISTEMUL DE FIȘIERE

- Trei părți majore

- Interfața cu utilizatorul și programatorul
 - Atribute, tipuri, operații, etc.
- Implementarea interfeței de către SO
 - Structuri de date și algoritmi
- Structura sistemelor de stocare secundare
 - Structura fizică
 - Nivelul cel mai apropiat de hw.

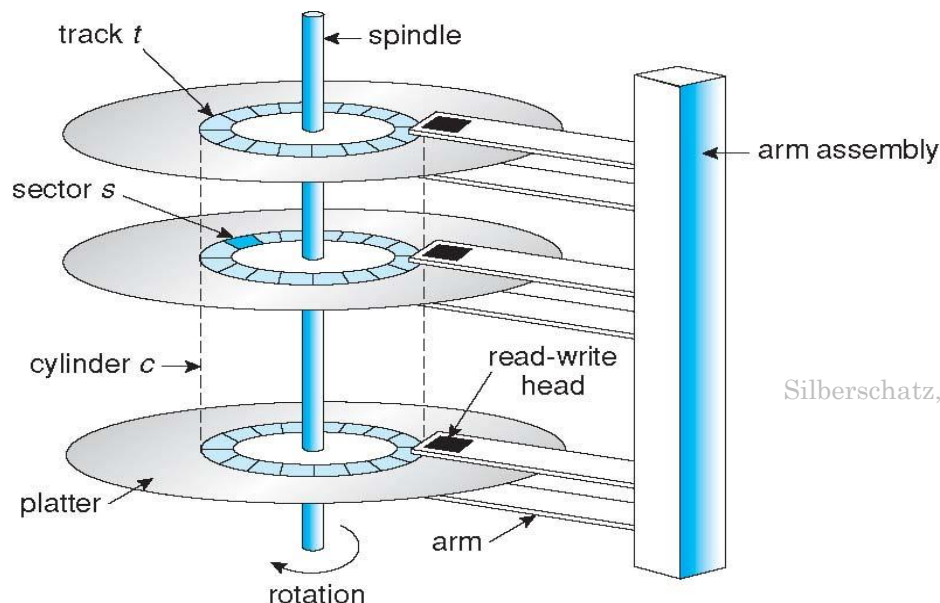
STRUCTURA DISPOZITIVELOR DE STOCARE

STRUCTURA SISTEMELOR DE STOCARE PERSISTENTĂ

○ Discuri magnetice

- Reprezintă majoritatea dispozitivelor de stocare secundare
- Structură relativ simplă
 - Fiecare **platan** are o formă circulară plată (ca un CD)
 - Ambele suprafețe acoperite cu un strat de material magnetic
 - **Capete de citire-scriere**
 - Atașate la ansamblul de brațe
 - Fiecare platan împărțit în **piste** și **sectoare**

Cilindru – mulțimea pistelor care se află la aceeași poziție a brațului de citire/scriere



Silberschatz, pg. 506

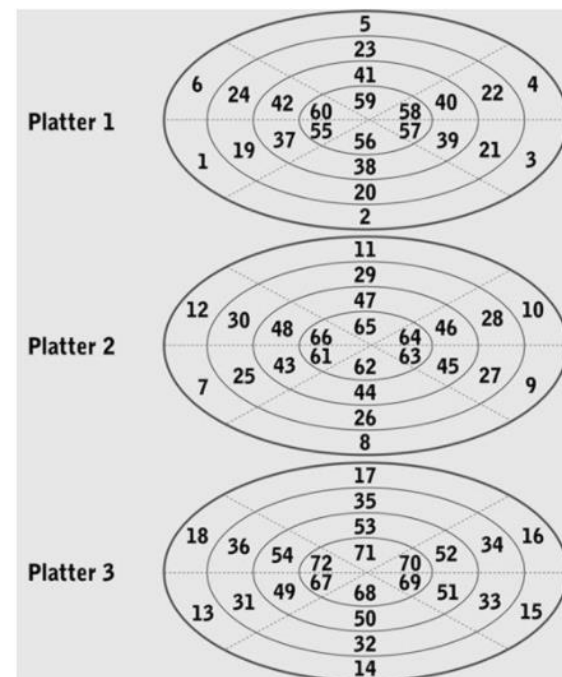
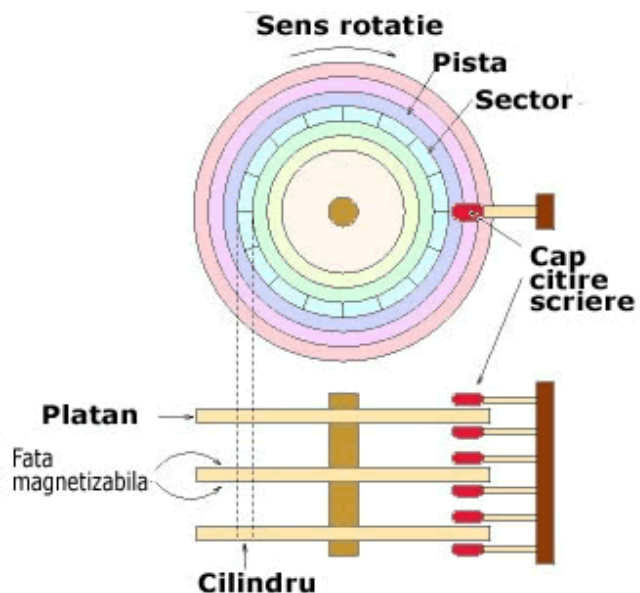
STRUCTURA DISCULUI MAGNETIC

Adresare

- Ca și un tablou unidimensional de blocuri logice – **sectoare** (blocuri de 512 octeți – mai nou ajung la 4 KB sau chiar 64 KB)
 - Se numerează sectoarele de la 0 la $n-1$ (pe un disc cu n sectoare)
 - **Spațiul de adrese** al discului

Mapare

- Blocurile logice sunt mapate pe sectoare în **mod secvențial**
 - Sectorul 0 este primul sector de pe prima pistă a cilindrului cel mai exterior
 - Maparea continuă în ordine pe pistă, apoi pe pistele aceluiași cilindru,
 - apoi pe restul cilindrilor



STRUCTURA DISCULUI MAGNETIC

○ Operații multi-sector

- SF citește / scrie blocuri de 4 KB (sau mai mult) în același timp
 - Cu garanția că scrierea unui sector (bloc de 512 B) este atomică

○ Platanii sunt plasați pe un ax central

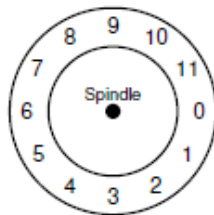
- La pornire un motor începe să învârtă axul
 - Viteză constantă – măsurată în **RPM** – rotații pe minut
 - Valori tipice: 7200 RPM - 15000 RPM
 - Ne interesează timpul necesar pentru o singură rotație
 - Ex. pt 10 000 RPM -> o singură rotație: 6 ms

○ În continuare

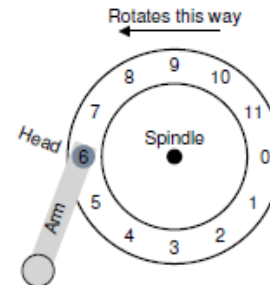
- Introducerea conceptelor pornind de la o structură simplistă

O UNITATE SIMPLĂ DE DISC

- O singură pistă cu 12 sectoare, fiecare de 512 B
 - Adresare: 0 – 11



A disk with just a single track

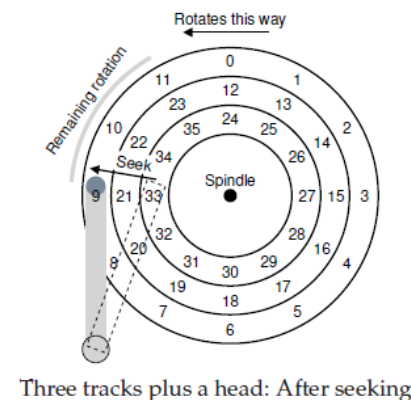
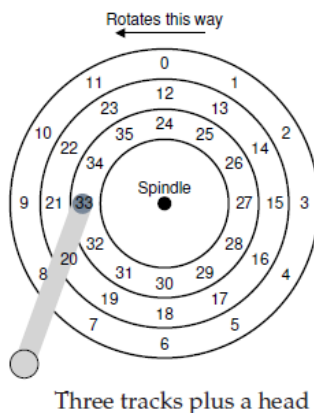


A single track plus a head

- Platanul se învâрте în jurul axului legat la motor
- Citirea / scriere se face la nivel de sector
 - Avem nevoie de un braț care se poate poziționa asupra sectorului
- **Latența rotațională**
 - Timpul necesar pentru ca sectorul dorit să se poziționeze sub capul de citire scriere
 - Valore medie: jumătate din timpul necesar unei rotații
 - Ex. viteză 7200 RPM -> ~ 4.2 ms
(max. $60 / 7200 \sim 8.33$ ms)

MAI MULTE PISTE

- Introducem mai multe piste – 3
 - Observăm **modul de aranjare** a sectoarelor pe piste
 - Poziția capului de cit./scr. pe sectorul 33 și dorim să citim sectorul 11
 - Se mută brațul discului pe pista corespunzătoare (cea mai exterioară)
 - operație numită căutare (seek)
 - **Timp de căutare**
 - Platanul este în rotație și peste două poziții se ajunge la sectorul 11
 - Timp de așteptare – **latența rotațională**
 - Se efectuează transferul
 - **Timp de transfer** = numărul de octeți de transferat / (viteza de rotație (sec) * numărul de octeți pe pistă)



MAI MULTE PISTE

○ Alte detalii

• Discuri multi-zonă

- Pistele exterioare conțin mai multe sectoare decât cele interioare

○ Zonă

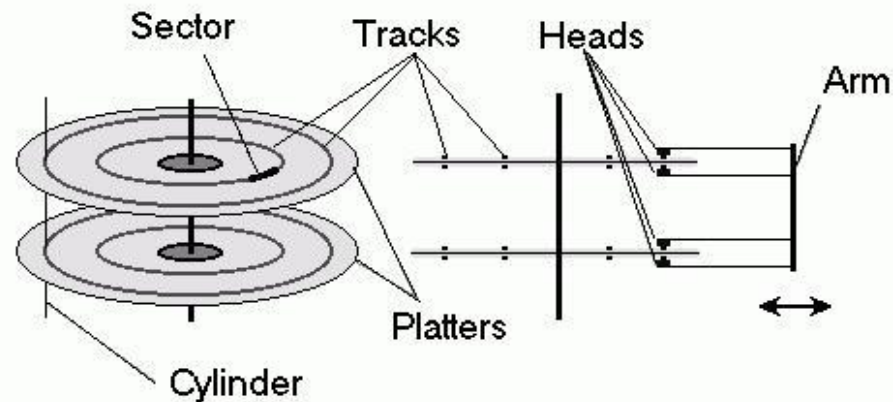
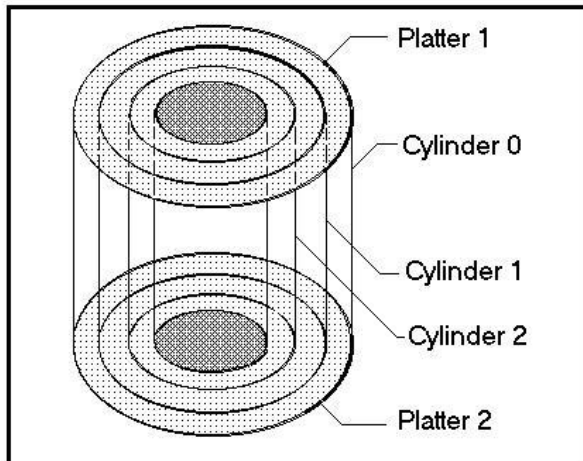
- set de piste consecutive cu număr egal de sectoare pe pistă

• Cache-ul discului (track buffer)

- Memorie (8 – 16 MB) în care se păstrează datele în tranzit
 - Citite de pe disc sau de scris pe disc
- La citire unitatea poate citi în buffer toate sectoarele de pe pistă
 - crește eficiența cererilor succesive pentru sectoare de pe aceeași pistă
- La scriere
 - Write-back caching sau immediate reporting
 - Write through
 - Write around

MAI MULTE PLATANE

- Două platane cu câte 3 piste
- Timpul de schimbare a capului
 - Timpul necesar pentru a schimba de la un cap la altul
 - De pe un platan pe altul
 - Brațul actuator mută toate capetele în mod sincronizat



TIMP DE ACCES

○ Două componente majore

○ Timp de căutare

- Timpul necesar pentru mutarea capului de cit. / scr. pe cilindrul corespunzător
 - 3 – 12 ms (aprox 7 ms pentru sisteme desktop)

○ Latența rotațională

- Timpul necesar rotirii discului pentru ca sectorul dorit să ajungă sub capul de citire / scriere
 - Val. medie $\frac{1}{2} * \frac{1}{(\text{RPM} / 60)}$

PERFORMANȚĂ

○ Exemplu comparativ

○ Disc cu

- viteza de rotație de 7500 RPM,
- timp mediu de căutare de 4 ms,
- dimensiunea unui sector de 512 B,
- 500 de sectoare pe pistă

○ Citim un fișier de aprox 1.28 MB (2500 de sectoare)

• Estimăm timpul total de transfer

○ Cazuri

• Organizare secvențială

- Fișier stocat compact în sectoare consecutive

• Alocare aleatoare (acces direct)

- Datele fișierului sunt distribuite pe disc

PERFORMANȚĂ

○ Caracteristicile discului

- viteza de rotație de 7500 RPM,
- timp mediu de căutare de 4 ms,
- dimensiunea unui sector de 512 B,
- 500 de sectoare pe pistă
- Citim un fișier de 1.28 MB (2500 de sectoare)

○ Organizare secvențială

- Timp pt citirea unei piste
 - Timp de căutare: 4 ms
 - Latența rotațională: 4 ms
 - Citirea a 500 de sectoare: 8 ms
- Timpul necesar următoarelor piste: 4 ms + 8 ms
 - Fără timp de căutare
- Citirea întregului fișier: $16 + (4 * 12) = 64 \text{ ms}$

○ Alocare aleatoare (acces direct)

- Timp pt citirea unui sector
 - Timp de căutare: 4 ms
 - Latența rotațională: 4 ms
 - Citirea unui sector: 0.016 ms
- Citirea întregului fișier: $2500 * 8.016 = 20.04 \text{ s}$

PLANIFICAREA DISCULUI

PLANIFICAREA DISCULUI

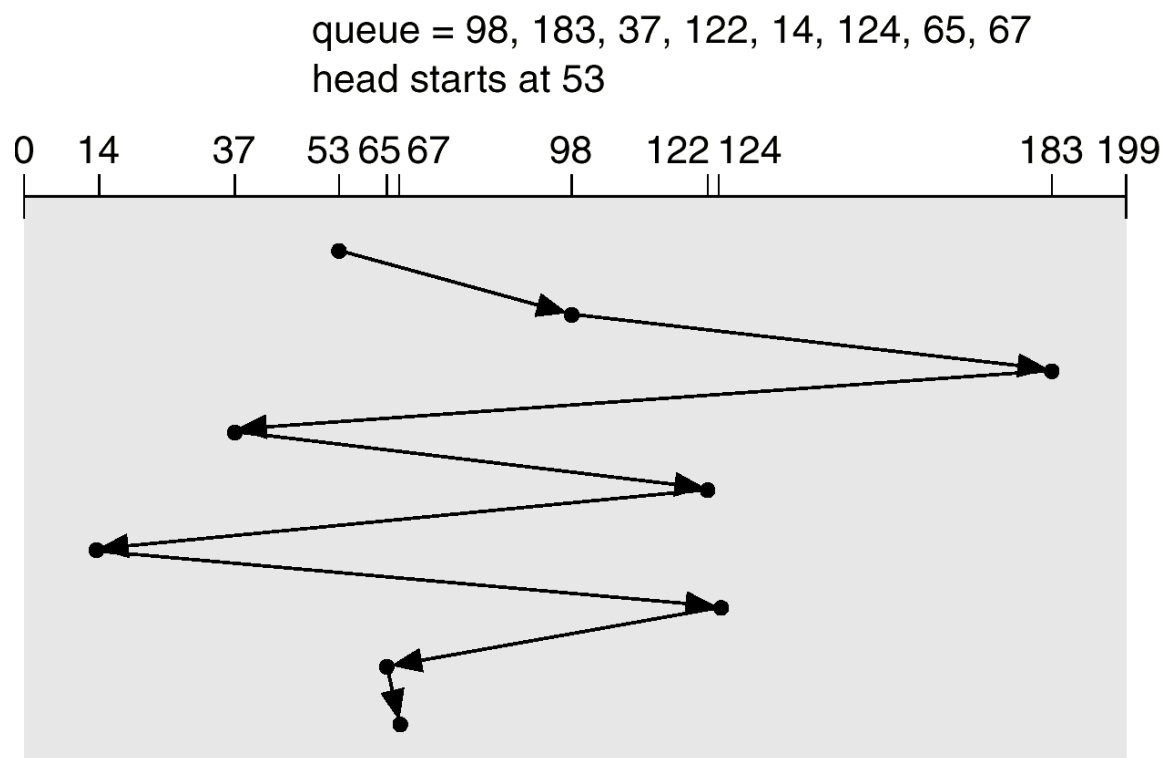
- SO este responsabil pentru **utilizarea eficientă** a hardware-ului
 - În cazul discului: acces rapid, lățime de bandă mare
- Scop
 - **Minimizare timpului de căutare**
 - Timpul de căutare este în legătură cu distanța de căutare
 - **Maximizarea** numărului de octeți transferați / timp total de la prima cerere până la ultimul transfer
 - **Lățime de bandă**
- SO menține o **coadă de cereri** de I/E către disc
 - În memoria controlorului de dispozitiv
 - Discurile inactivate pot deservi cererea imediat însă
 - Discurile ocupate necesită o coadă de lucru
- Optimizările au sens doar în cazul existenței cozii de cereri I/E
 - **Alegerea ordinii** de efectuare a operațiilor **de I/E**

PLANIFICAREA DISCULUI

- Ilustrarea planificării pentru coada de cereri (pistele discului: numerotate 0 - 199)
 - 98, 183, 37, 122, 14, 124, 65, 67
- La început capul de citire se află la cilindrul 53

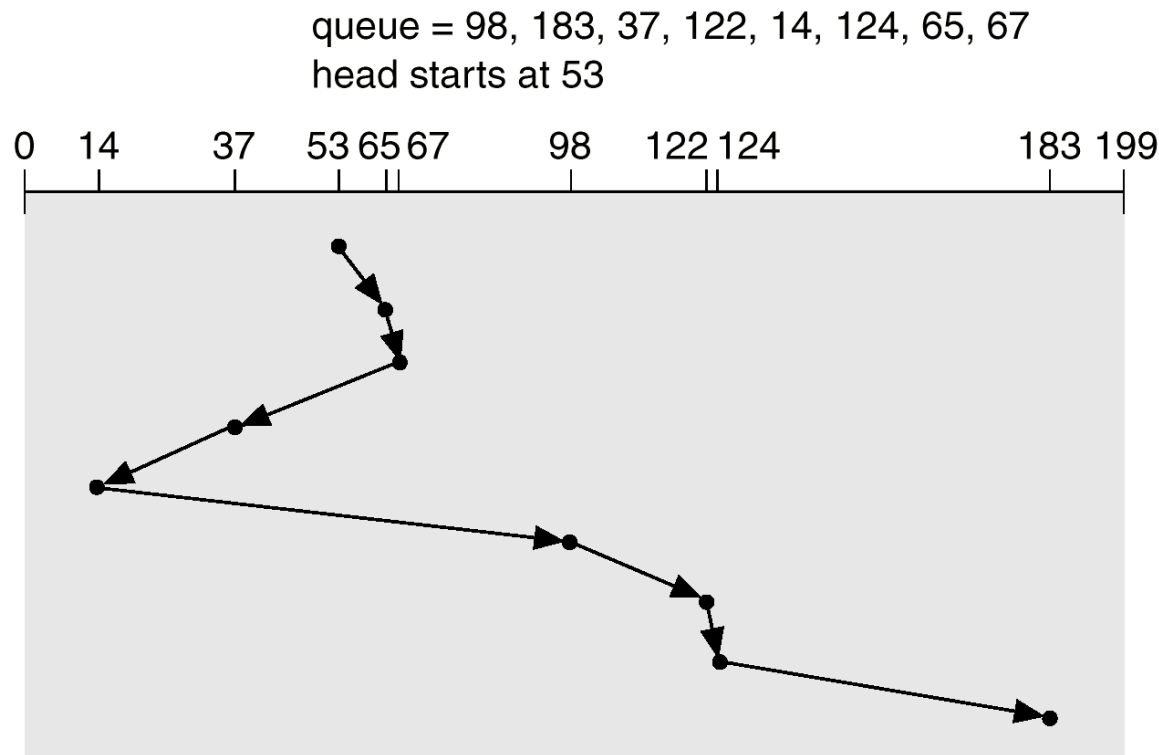
FCFS – FIRS-COME, FIRST-SERVED

- Metoda cea mai simplă
 - Corectitudine
 - Timp de căutare mare
- Număr de piste traversate: 640



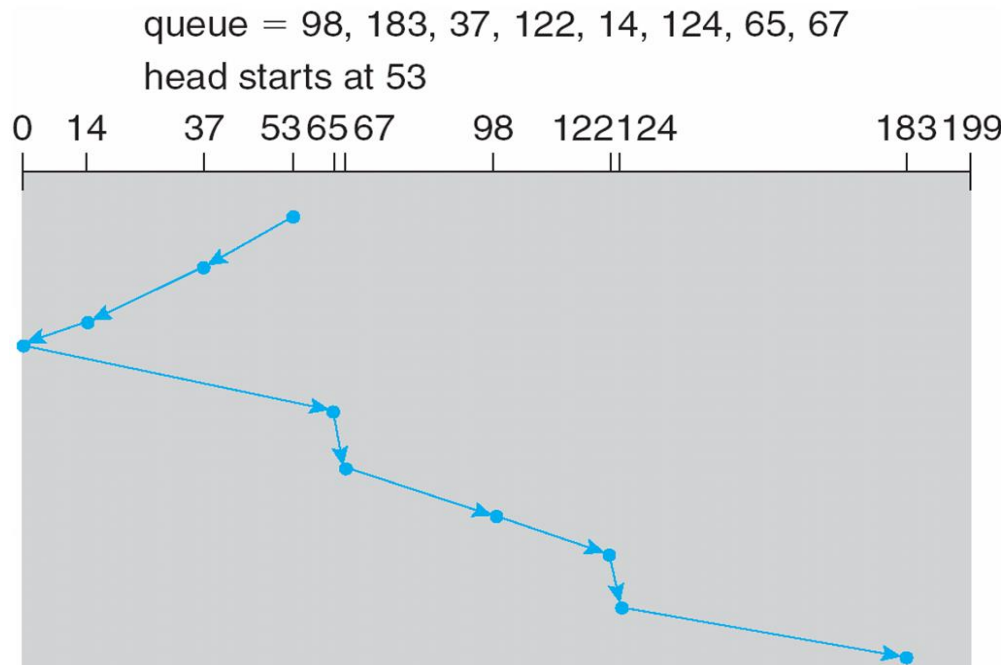
SSTF – SHORTEST-SEEK-TIME-FIRST

- Selectarea cererii care introduce **cel mai mic timp de căutare**
 - Analogie cu SJF de la procese – poate apare înfometarea
- Număr de piste traversate: 236 (aprox 1/3 din FCFS)



SCAN – NUMIT ȘI ASCENSOR

- Brațul capului traversează pistele **dintr-un capăt al discului către celălalt - și apoi invers**
 - Presupunem că direcția este către cilindrul 0
 - La întoarcerea capului sunt puține cereri imediat în față și mai multe în celălalt capăt
- Număr de piste traversate: 236

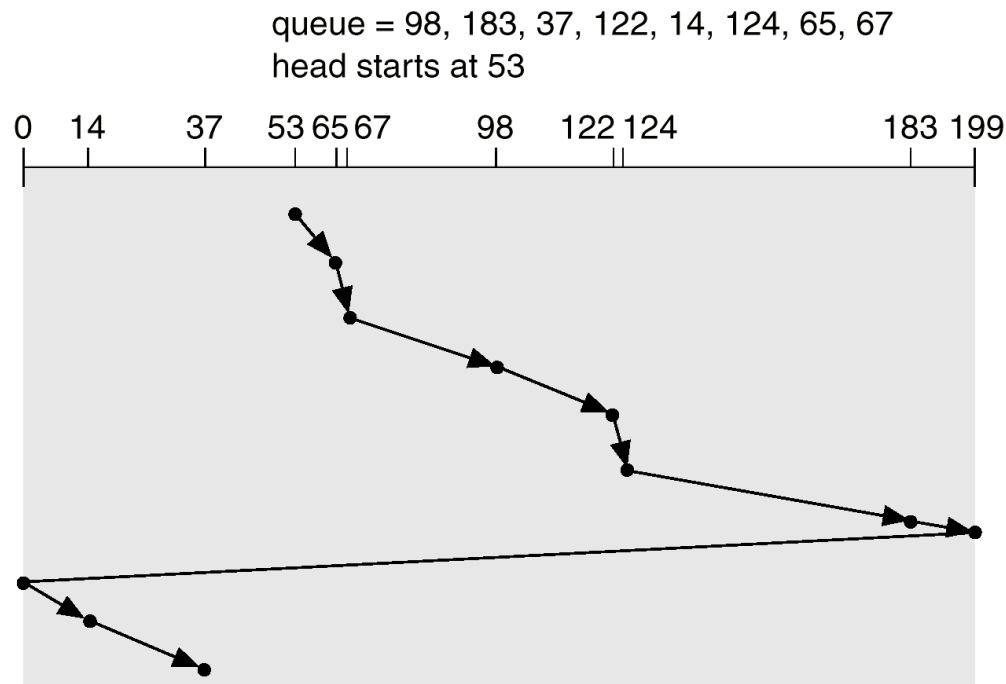


Silberschatz, pg. 514

C-SCAN

○ Variantă a planificării Scan unde

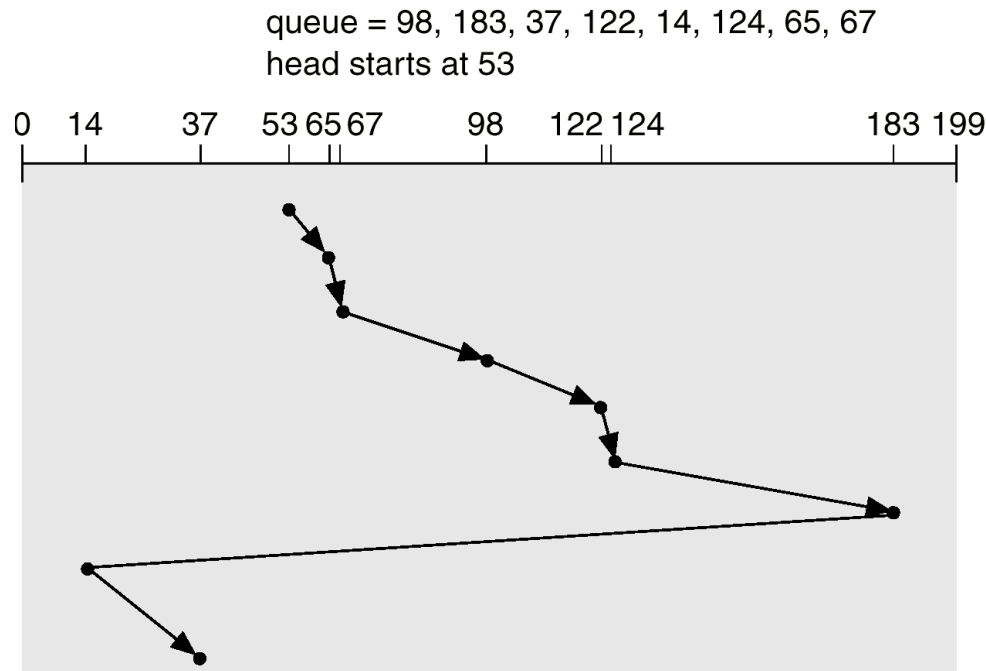
- brațul capului traversează pistele dintr-un capăt al discului către celălalt – într-o singură direcție
 - După ce brațul ajunge într-un capăt al discului revine apoi imediat la celălalt capăt și traversează discul în aceeași direcție
 - Abordare asemănătoare unei liste circulare
- Număr de piste traversate: $(65-53)+(67-65)+(98-67)+(122-98)+(124-122)+(183-124)+(199-183)+\textcolor{blue}{(199-0)}+(14-0)+(37-14) = \textcolor{red}{183} + 199$
 - – (199-0) nu se considera de regula (revenirea capului nu se consideră la piste traversate)



Silberschatz, pg. 514

LOOK ȘI C-LOOK

- Similar cu Scan și C-Scan dar
 - brațul capului traversează piste în ordine **până la ultimele cereri** din fiecare direcție
 - Număr de piste traversate : $183 - 53 + 37 - 14 + (183 - 14)$
 - – 153 dacă revenirea capului nu se consideră la numărul de piste traversate



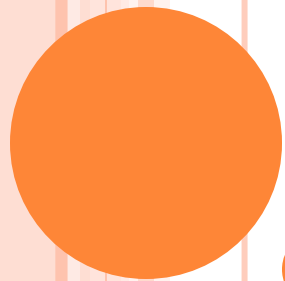
Silberschatz, pg. 515

SELECTAREA UNUI ALGORITM DE PLANIFICARE

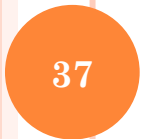
- SSTF foarte popular
 - performanță mult mai bună decât la FCFS
- SCAN și C-SCAN au performanță mai bună pentru sisteme cu încărcare mare la nivelul discului
 - Nu creează probleme de înfometare
- Performanța depinde de
 - Numărul și **tipul cererilor**
 - **Metoda de alocare** a fișierelor
 - Alocare contiguă vs. alocare prin înlănțuire resp. indexare
 - Localizarea **blocurilor de control** al fișierelor
 - Folosirea unui cache pentru directoare si BCF-uri ajută la reducerea mișcărilor necesare ale brațului
- Algoritmii de planificare a discului – modul separat în SO
 - Pentru a putea fi înlocuit la nevoie
 - Implicit SSTF sau LOOK (C-LOOK)
- Aici am luat în considerare doar **timpul de căutare**
 - Însă latența rotațională poate ajunge la valori similare cu timpul de căutare mediu

SELECTAREA UNUI ALGORITM DE PLANIFICARE

- Latența rotațională este greu de calculat de SO
 - Unități de disc moderne nu dezvăluie locația fizică a blocurilor logice
 - Producătorii includ în controlerul de dispozitiv **algoritmi de planificare** care optimizează atât timpul de căutare cât și latența rotațională
- Dacă performanța I/O ar fi singurul considerent
 - SO ar preda responsabilitatea planificării discului însuși
- SO decide să implementeze propriul algoritm de planificare
 - Pentru că în unele cazuri cererile au **priorități diferite**
 - Paginare la cerere are prioritate mai mare decât o operație de I/O din partea aplicației
 - Pentru a **garanta ordinea** unor scrieri pe disc – pt. fiabilitate



RAID



37



RAID

- În lucrul cu discul de multe ori ne dorim să avem acces la
 - Discuri mai mari, mai rapide, mai fiabile
- Soluție: **RAID – Redundant Array of Independent (Inexpensive) Disks**
 - Mai multe discuri care operează independent și paralel
 - Viziunea SO: un singur disc
 - **Interfața RAID**
 - SF percepe RAID-ul ca pe un disc mare și fiabil
 - Se prezintă ca un tablou liniar de blocuri - fiecare poate fi citit și scris
 - De regulă este o cutie hw separată cu interfață standard (ex. SCSI, SATA)
 - În interior **structură complexă** (asemănător unui mini-calculator specializat)
 - Mai multe discuri
 - Memorie volatilă și nevolatilă
 - Procesor specializat pt gestiunea sistemului de stocare

RAID

○ Avantaje

○ Performanță

- Acces paralel la mai multe discuri → deservirea mai multor cereri de I/E în paralel (dacă datele cerute sunt pe discuri diferite)
 - îmbunătățirea ratei de citire/scriere și a timpului de transfer

○ Capacitate

- Permite volum de stocare mare

○ Fiabilitate

- Prin integrarea unui grad de redundanță – poate tolera pierderea unui disc
 - Proiectate să detecteze și să revină din anumite defecte

• Toate acestea furnizate în mod transparent

- Nu necesită modificări asupra altor componente ale sistemului

RAID

○ Fiabilitate prin **redundanță**

- Stocarea informației suplimentare care poate fi utilizată în cazul căderii unui disc pentru a recupera informația pierdută
 - Chiar dacă un disc cade, datele nu sunt pierdute
- NVRAM – Nonvolatile RAM cu rol de write-back cache
 - Cache nevolatil adăugat RAID-ului pentru a proteja scrierile în fața penelor de curent

• **Mirroring**

- Un disc logic constă din două discuri fizice
 - Scrierea se face pe ambele – redundanță

○ Performanță prin **paralelism**

- La mirroring: accesul paralel la două discuri în paralel duce la dublarea ratei de deservire a citirilor
 - Rata de transfer a fiecărei citiri este aceeași, dar numărul de citiri pe unitate de timp se dublează

• **Data striping**

- Fiecare unitate (bloc, sector, etc.) din fișier este distribuit pe discuri consecutive (alocate round-robin)
 - La nivel de bit – fiecare bit din fiecare octet scris pe alt disc (bit-level striping)
 - La nivel de octet
 - **La nivel de bloc** – în toate nivelurile RAID utilizate practic

- Reduce timpul de răspuns la accesul unui volum mare de date
- Îmbunătățește rata de transfer pentru multiple accesări scurte (load balancing)

NIVELURI RAID



(b) RAID 1: mirrored disks.



(c) RAID 2: memory-style error-correcting codes.



(d) RAID 3: bit-interleaved parity.



(e) RAID 4: block-interleaved parity.



(f) RAID 5: block-interleaved distributed parity.



(g) RAID 6: P + Q redundancy.

Copie
secundară

Coduri
corectoare
de erori

EVALUARE RAID

○ Diferitele niveluri RAID au caracteristici diferite

○ Metrice de evaluare

• Capacitate

- Dacă avem N discuri cu câte B blocuri fiecare, care este capacitatea disponibilă utilizatorilor pentru stocare?
 - Dacă nu avem redundanță - RAID nivelul 0 -> $N * B$
 - Dacă avem oglindire (mirroring) - RAID nivelul 1 -> 1
Nivelul 1 pur – oglindirea unui disc
 - Alte variante se încadrează undeva între aceste valori

• Fiabilitate

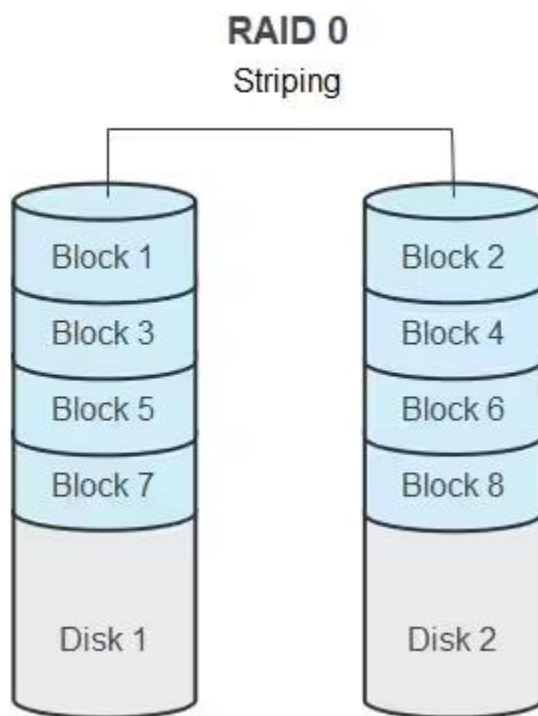
- Câte disk fault-uri poate tolera RAID-ul?
 - Dacă considerăm că disk fault = căderea întregului disc

• Performanța

- Mai greu de evaluat
- Trebuie să țină cont de volumul de lucru

RAID – NIVELUL 0: STRIPING

- Nu include redundanță – nu este un RAID adevărat
- Datele sunt distribuite pe toate discurile din RAID – striping
 - Distribuie blocurile între discuri în mod round-robin
 - Blocurile (strip) de pe aceeași linie (ex. 0, 1, 2 și 3) formează un **stripe**
- Când performanța și capacitatea sunt mai importante decât fiabilitatea



RAID – NIVELUL 0: STRIPING

- Nu include redundanță – nu este un RAID adevărat
- Datele sunt distribuite pe toate discurile din RAID – striping
 - Distribuie blocurile între discuri în mod round-robin
 - Blocurile (strip) de pe aceeași linie (ex. 0, 1, 2 și 3) formează un **stripe**
- Când performanța și capacitatea sunt mai importante decât fiabilitatea

Disk 0	Disk 1	Disk 2	Disk 3
0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

Figure 38.1: RAID-0: Simple Striping

Disk 0	Disk 1	Disk 2	Disk 3	
0	2	4	6	chunk size: 2 blocks
1	3	5	7	
8	10	12	14	
9	11	13	15	

Figure 38.2: Striping with a Bigger Chunk Size

MAPAREA BLOCURILOR LOGICE LA LOCAȚIA FIZICĂ

- La apariția unei cereri de acces la un bloc logic, RAID-ul trebuie să identifice care disc fizic și ce offset să acceseze

- La nivele RAID simple – maparea este simplă

- RAID nivelul 0

- Adresa blocului logic: A
 - Numărul de discuri din RAID: N
 - $Nr_disc: A \% N$
 - Offset: A / N

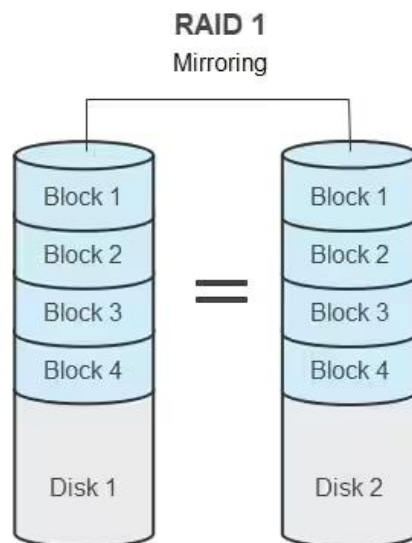
Disk 0	Disk 1	Disk 2	Disk 3
0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

Figure 38.1: RAID-0: Simple Striping

- Ex. Adresa logică $A=14$ și Numărul de discuri din RAID: 4
 - $Nr_disc: 14 \% 4 = 2$ (discul numărul 2 - al treilea disc)
 - Offset: $14 / 4 = 3$ (blocul numărul 3 - al patrulea bloc)

RAID – NIVELUL 1 - **MIRRORING**

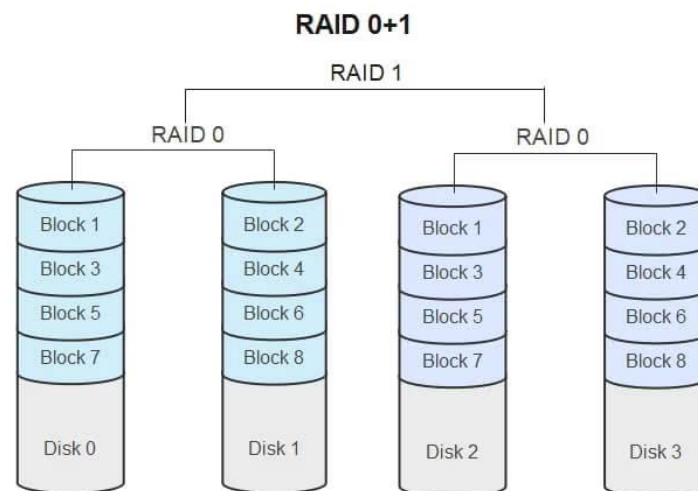
- Redundanță prin **date duplicat**
 - Fiecare copie pe disc separat
 - Citirea se poate face de pe oricare disc care conține data (care implică timp mai scurt)
 - Scrierea trebuie efectuată pe ambele discuri (poate fi efectuată în paralel)
- Dacă discul eșuează datele pot fi accesate de pe discurile secundare
- Dezavantaj: **cost mare**
 - Spațiu fizic (cel puțin) dublu față de spațiul logic
- Fiabilitate
 - Tolerază eșecul oricărui disc (chiar si mai multe discuri până rămâne cel puțin o copie a fiecărei date)



RAID 01 ȘI RAID 10

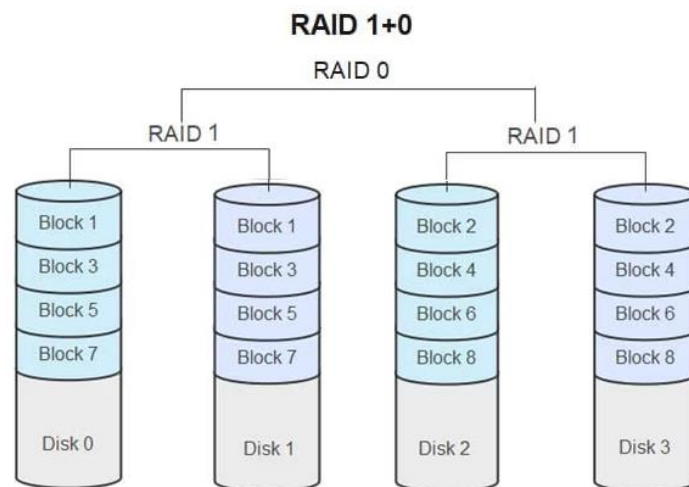
- RAID 01 (RAID 0+1) – mirror of stripes

- Mai întâi se face RAID 0 (striping)
 - datele sunt împărțite pe mai multe discuri
- Apoi acel grup este oglindit (RAID 1) pe un alt grup identic



- RAID 10 – stripe of mirrors

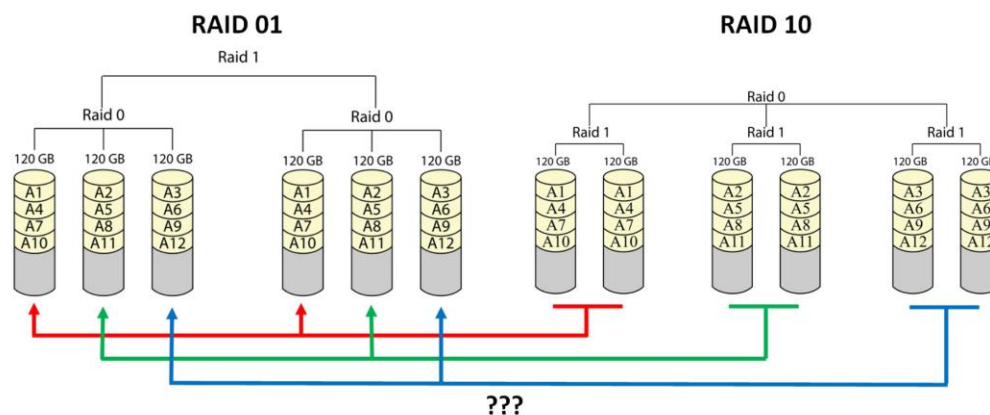
- Se creează mai întâi perechi RAID 1 (mirroring)
- Apoi aceste perechi sunt combinate în RAID 0 (striping)
- RAID 10 este mai tolerant la erori decât RAID 01, deci este utilizat pe scară largă



RAID 01 ȘI RAID 10

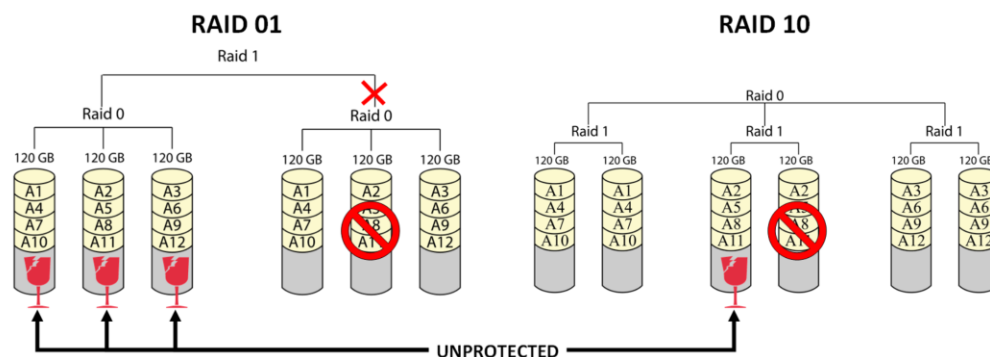
Capacitate și performanță similare

- capacitate de stocare $n/2$
- pot supraviețui întotdeauna unei unități defecte fără pierderi de date
- maximul teoretic pentru unitățile defecte pentru ambele este $n/2$



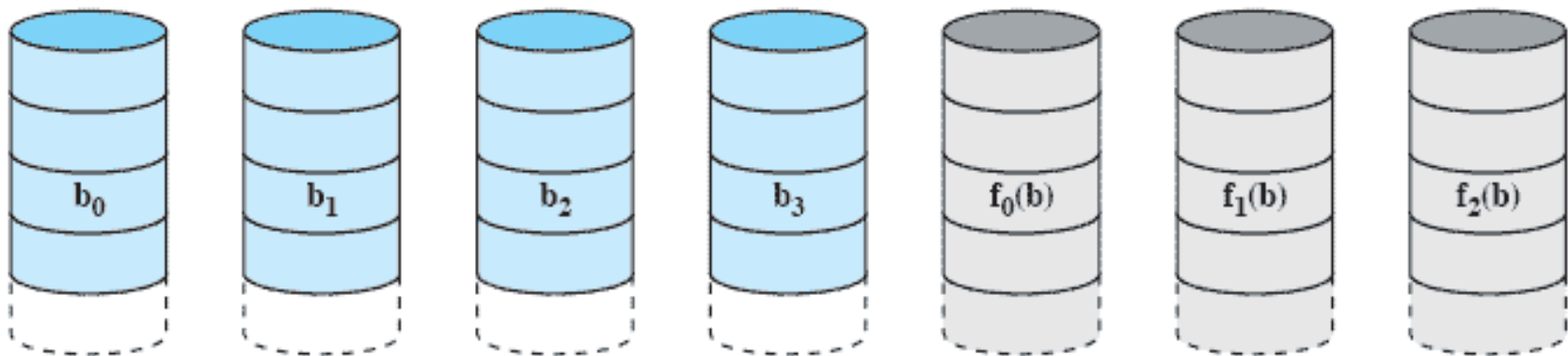
RAID 10 performanță superioară în caz de eroare

- Dacă un disc eșuează
 - RAID 01 lasă jumătate din unități neprotejate
 - pierde o întreagă bandă RAID 0
 - RAID 10 își lasă doar partenerul
 - celelalte oglinzi nu sunt afectate



RAID – NIVELUL 2

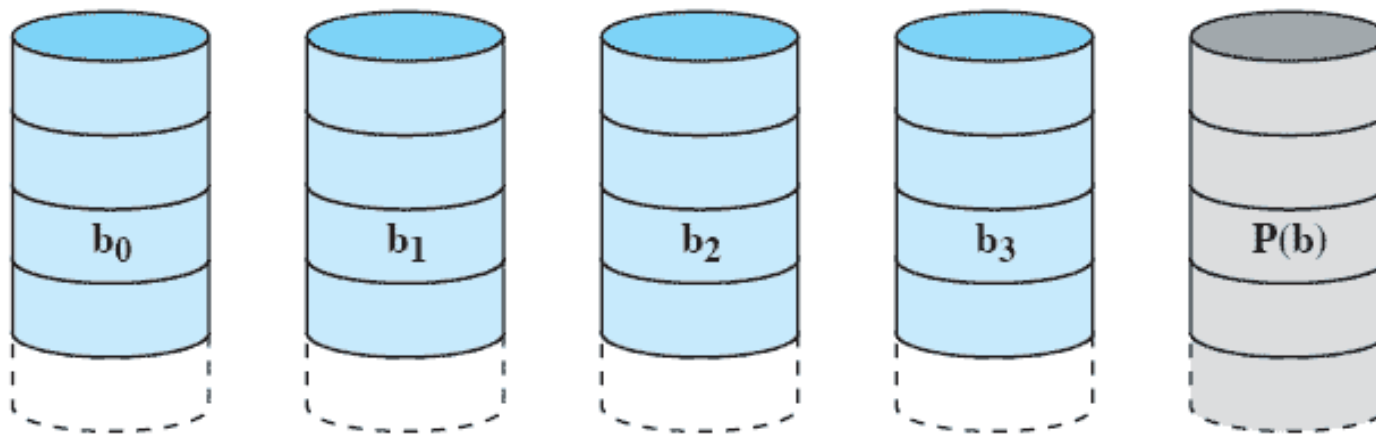
- Tehnică de **acces paralel**
 - Mișcarea brațelor din discuri este sincronizată
 - Toate discurile din RAID participă la deservirea fiecărei cereri de I/O
 - De regulă nu poate servi mai multe cereri simultan
- Data striping la nivel de **bit, octet sau cuvânt**
- Redundanță prin **coduri corectoare de erori** (ec. Coduri Hamming)
 - Numărul de discuri de redundanță proporțional cu logaritmul numărului de discuri de date
 - Poate corecta erori de un bit
 - Poate detecta erori de doi biți
- Extrem de rar folosit – cost mare + noile discuri includ deja coduri corectoare de erori
 - Recomandat doar în medii în care apar multe erori de disc



(c) RAID 2 (redundancy through Hamming code)

RAID – NIVELUL 3

- Similar cu nivelul 2 dar cu un sigur disc redundant
- **Acces paralel**, date distribuite în bucăți mici (biți sau octeți)
- Calculează **bit de paritate** pentru setul biților de pe aceeași poziție
- Furnizează **rată de transfer mare**
 - Dar doar o **singură cerere** deservită odată

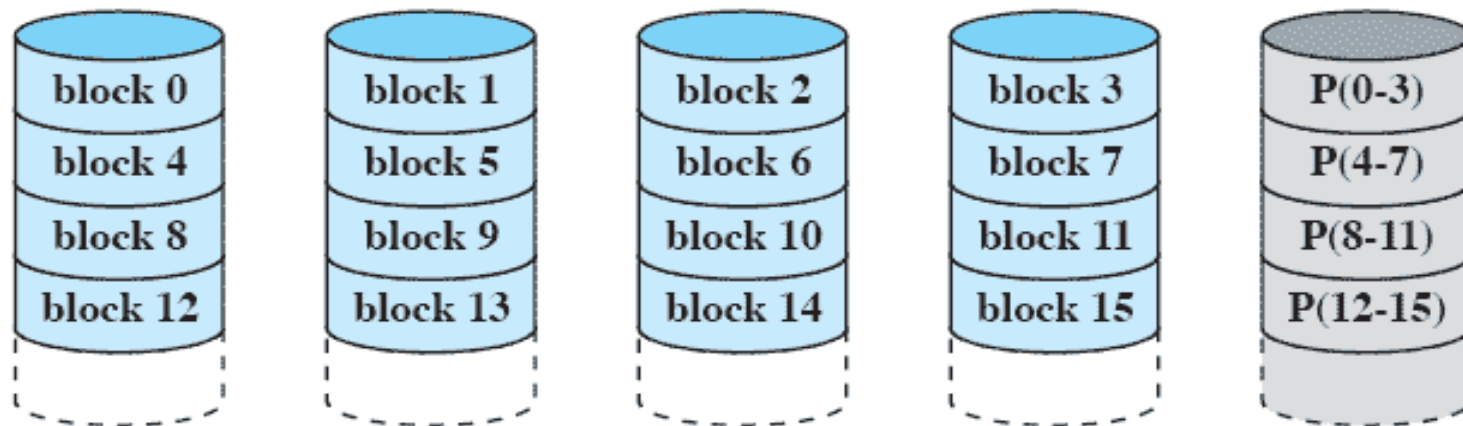


(d) RAID 3 (bit-interleaved parity)

RAID – NIVELUL 4

- Data striping la nivel mai mare (**blocuri**)
- Tolerează eșecul unui singur disc de date
- Se calculează **paritatea** bit-cu-bit pentru blocurile corespunzătoare de pe discuri
 - Bitul de paritate se stochează pe discul de paritate
- **Penalizare** la scrierea unui volum mic de date
- Discul de paritate poate provoca **gâtuire**

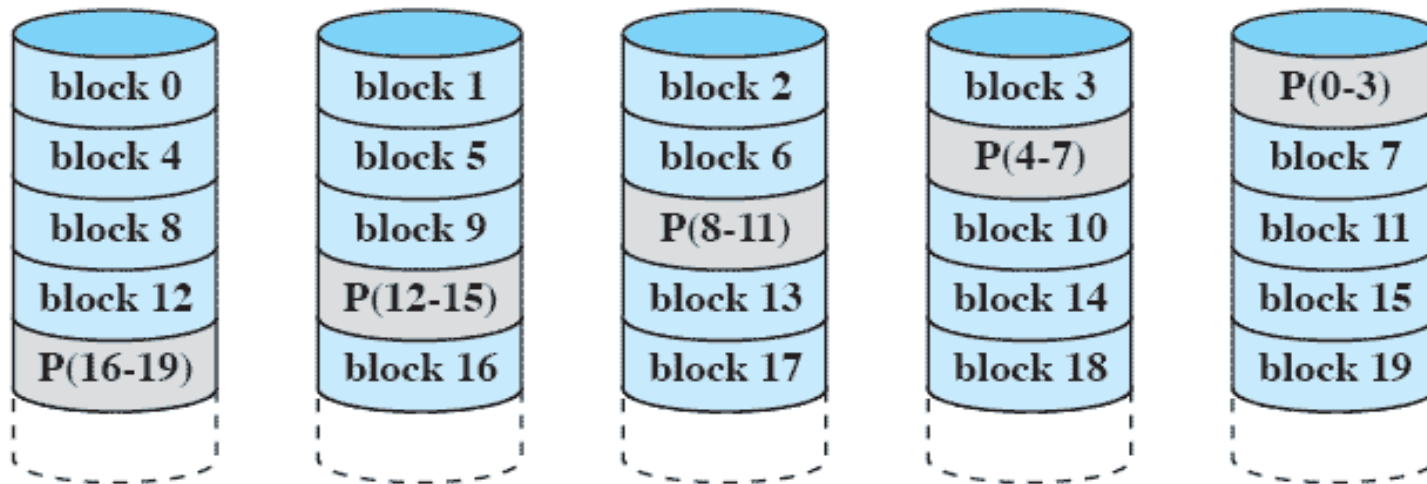
C0	C1	C2	C3	P
0	0	1	1	$XOR(0,0,1,1) = 0$
0	1	0	0	$XOR(0,1,0,0) = 1$



(e) RAID 4 (block-level parity)

RAID – NIVELUL 5

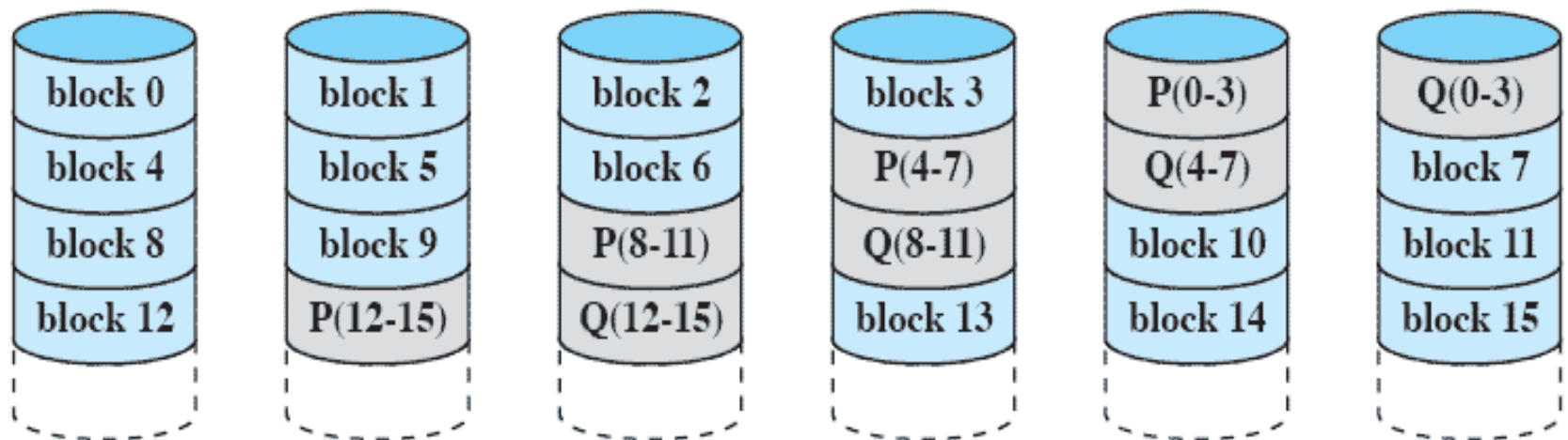
- Asemănător cu 4 dar **distribuie blocurile de paritate** pe toate discurile
 - În general prin alocare round-robin
- Nu mai apare problema gâtuirii (de la niv. 4)
- Cel mai frecvent utilizat sistem RAID cu paritate



(f) RAID 5 (block-level distributed parity)

RAID – NIVELUL 6

- Se efectuează **două calcule de paritate** și valorile sunt stocate în blocuri separate pe discuri diferite
 - **P și Q** algoritmi de verificare diferiți (unul XOR – ca și la niv 4 și 5)
 - Permite recuperare chiar și în situația în care eșuează două discuri
- Furnizează disponibilitate foarte mare
- **Penalizare** substanțială la scriere
 - Fiecare scriere afectează și două blocuri de paritate



(g) RAID 6 (dual redundancy)

NIVELURI RAID

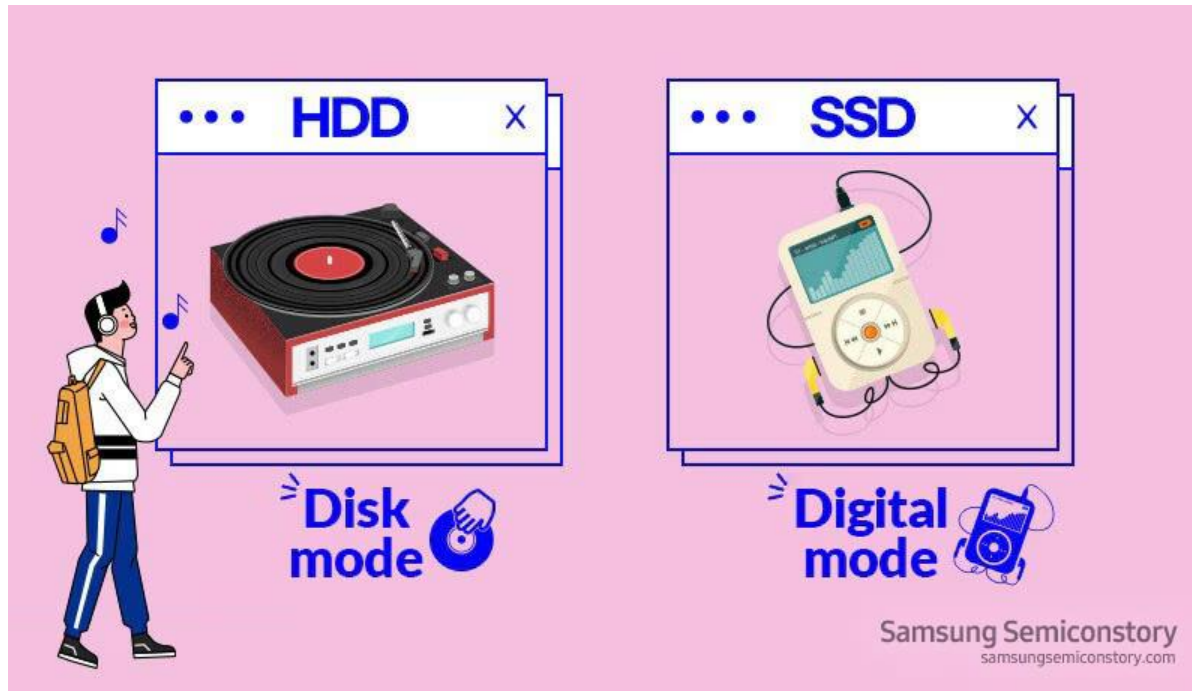
Category	Level	Description	Disks required	Data availability	Large I/O data transfer capacity	Small I/O request rate
Striping	0	Nonredundant	N	Lower than single disk	Very high	Very high for both read and write
Mirroring Parallel access	1	Mirrored	$2N$	Higher than RAID 2, 3, 4, or 5; lower than RAID 6	Higher than single disk for read; similar to single disk for write	Up to twice that of a single disk for read; similar to single disk for write
	2	Redundant via Hamming code	$N + m$	Much higher than single disk; comparable to RAID 3, 4, or 5	Highest of all listed alternatives	Approximately twice that of a single disk
	3	Bit-interleaved parity	$N + 1$	Much higher than single disk; comparable to RAID 2, 4, or 5	Highest of all listed alternatives	Approximately twice that of a single disk
Independent access	4	Block-interleaved parity	$N + 1$	Much higher than single disk; comparable to RAID 2, 3, or 5	Similar to RAID 0 for read; significantly lower than single disk for write	Similar to RAID 0 for read; significantly lower than single disk for write
	5	Block-interleaved distributed parity	$N + 1$	Much higher than single disk; comparable to RAID 2, 3, or 4	Similar to RAID 0 for read; lower than single disk for write	Similar to RAID 0 for read; generally lower than single disk for write
	6	Block-interleaved dual distributed parity	$N + 2$	Highest of all listed alternatives	Similar to RAID 0 for read; lower than RAID 5 for write	Similar to RAID 0 for read; significantly lower than RAID 5 for write

N = number of data disks; m proportional to $\log N$

Stallings, pg. 496

DE LA HDD LA SSD

- An HDD is to vinyl as an SSD is to MP3!



- SSD = performanță, latență mică, tehnologie modernă
- HDD = capacitate mare, cost mic, tehnologie matură
- Diferența majoră: mecanic vs electronic!

SSD – SOLID STATE STORAGE

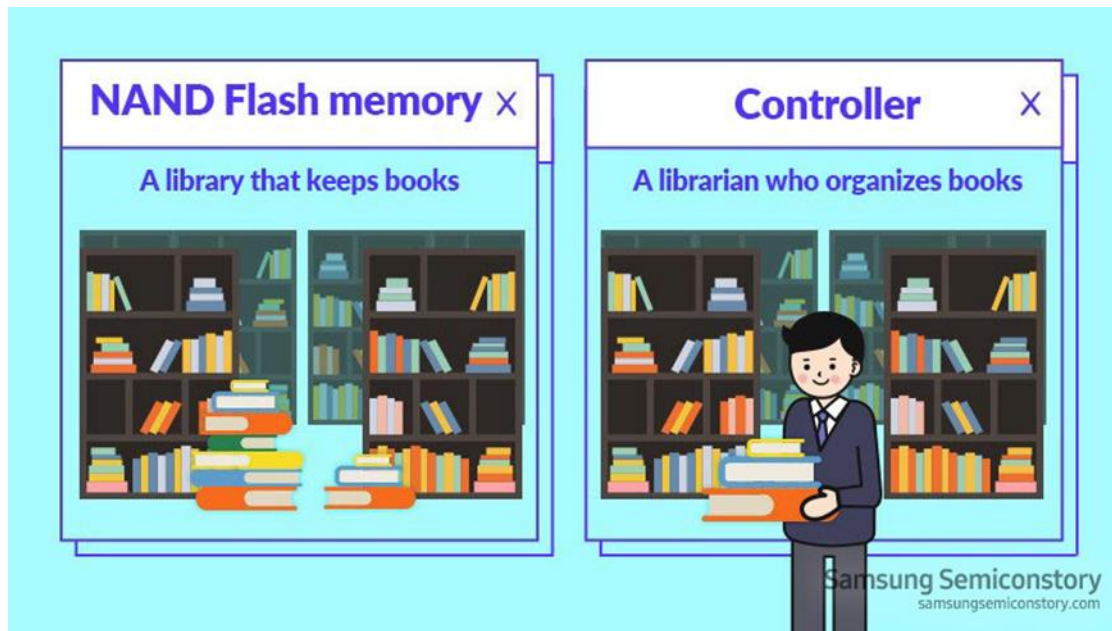
- Dispozitive de stocare persistentă
- Construite din microcipuri – similar cu memoria internă
 - Folosesc chipuri de memorie NAND flash
- Față de HDD
 - Nu au părți mecanice mobile
 - Timp de acces mai mic
 - Durabilitate mai mare
 - Mai rezistente la șocuri mecanice, vibrații și variații de temperatură
 - Preț mai mare
 - Capacitate mai mică
 - Cicluri de scriere limitate
 - Recuperare mai dificilă a datelor



<https://www.techchef.in/ssd-vs-hdd-discover-the-better-storage-device/>

SSD – SOLID STATE STORAGE

- Conține un controler electronic inteligent – un procesor
 - Detecția erorilor
 - Detectarea și distribuirea nivelului uzurii (Wear leveling)
 - Realocarea blocurilor de memorie corupte
 - Colectarea resturilor (Garbage collection)
 - eventual și criptare

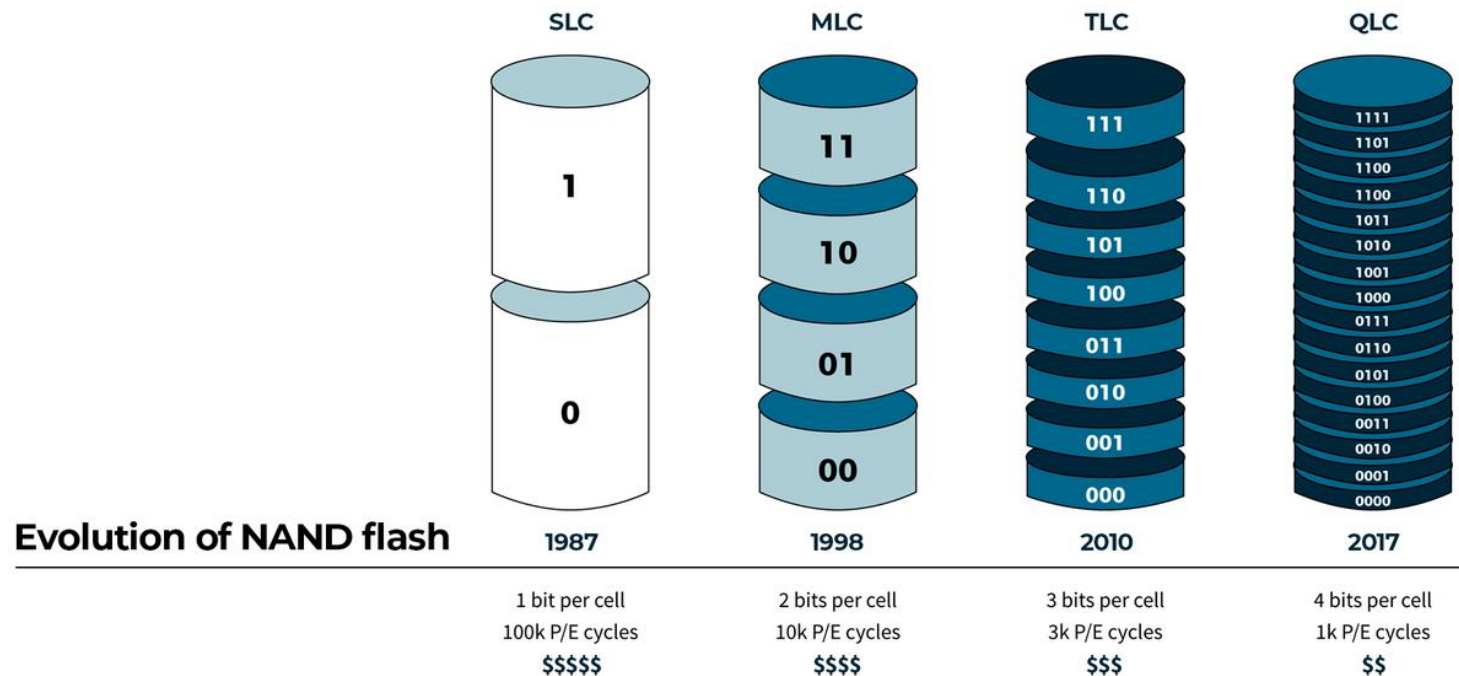


TIPURI DE MEMORIE NAND

○ Datele sunt salvate în celule

- Fiecare celulă poate salva un anumit număr de biți
- Cu cât sunt mai mulți biți, cu atât SSD-ul este mai ieftin și mai mare, dar mai lent și mai puțin durabil

SLC = performanță maximă (rar folosit)
MLC = echilibru (mai vechi)
TLC = standardul actual
QLC = capacitate mare, buget



BLOCURI ȘI PAGINI

○ Structurare

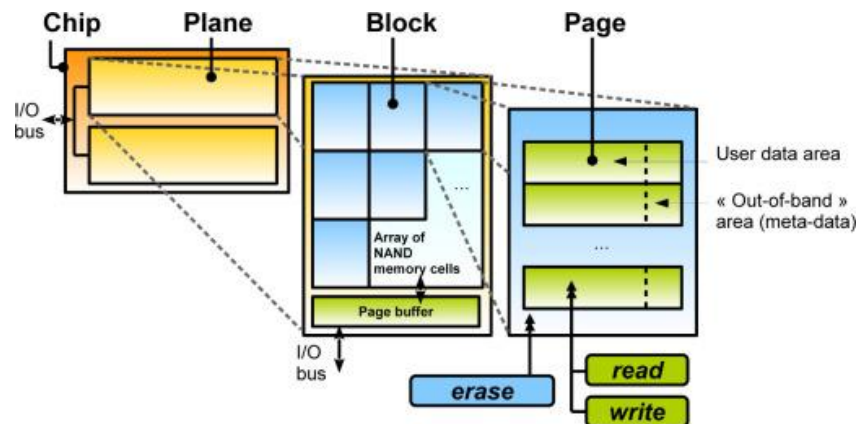
- Celulele formează pagini
- Paginile formează blocuri

○ Pagina

- Este **cea mai mică unitate de scriere**
 - Dimensiune tipică: **4 KB – 16 KB**
 - Atenție: SSD-ul scrie date **doar în pagini**, nu la nivel de bit individual
- Poți **citi și scrie** la nivel de pagină
- Dar NU poți șterge o singură pagină

○ Bloc

- Este un grup de pagini (de obicei **64 – 256 pagini**)
 - Dimensiune tipică: **256 KB – câțiva MB**
- Ștergerea datelor se face **doar la nivel de bloc**



BLOCURI ȘI PAGINI

○ Problema la scriere - Write Amplification

- Scrierile mici sunt mai lente → SSD-ul face mai multă muncă intern
 - SSD-urile folosesc cache ca să compenseze
 - SSD-urile mai bune au controlere mai inteligente

○ Exempu: Modificarea unui fișier mic

- SSD-ul NU poate suprascrie direct pagina
- Trebuie:
 - să copieze datele bune într-un alt bloc
 - să modifice pagina
 - să șteargă blocul vechi

○ Soluție

○ **Garbage Collection**

- mută datele utile
- eliberează blocuri întregi

○ **TRIM**

- sistemul de operare indică ce date nu mai sunt necesare
- SSD-ul le poate șterge eficient

○ Impact asupra duratei de viață

- Fiecare ștergere de bloc = uzură
- Soluții
 - **Wear leveling**
 - limită de scrieri (TBW)

Pagina = unitate de scriere
Blocul = unitate de ștergere
Nu poți modifica direct datele
SSD-ul mută date → optimizare automată

SCRIEREA DATELOR

- Scriere în blocuri
- Nu există fragmentare relevantă
- Folosește tehnici speciale:
 - Wear leveling (distribuie uzura)
 - Garbage collection
 - TRIM (ștergere eficientă a datelor)

○ IOPS

- HDD: ~100–200 IOPS
- SSD: 50.000 – 1.000.000+ IOPS

○ Interfețe

- SATA (limitat la ~550 MB/s)
- NVMe (PCIe) – mult mai rapid
 - Exemplu:
 - SATA III: max ~600 MB/s
 - PCIe 3.0 NVMe: ~3500 MB/s
 - PCIe 4.0 NVMe: ~7000 MB/s

- SSD-ul este de **10–50x** mai rapid decât HDD la acces aleator

- Diferența se simte cel mai mult la:
 - pornirea sistemului
 - deschiderea aplicațiilor
 - copierea fișierelor mici

DISPOZITIVE DE STOCARE TERȚIARE

○ Caracteristica definitorie

- Cost redus
- Mediu detașabil
 - Ex. CD-ROM, floppy, discuri electronice, etc.

○ Discuri detașabile

- Floppy
- Discuri magnetice detașabile
 - Aproape la fel de rapide ca și hard discurile dar sunt mai expuse avarierii
- Discuri optice
 - Read-write disks
 - WORM – Write-Once, Read-many-times
 - Read-only disks
 - CD-ROM, DVD-ROM

○ Benzi

- Cost mai redus decât la discuri, dar accesul direct este mult mai lent
- Mediu potrivit pentru backup, volum mare de date